RWANDA POLYTECHNIC

KIGALI COLLEGE

ACADEMIC YEAR: 2026-2027

DEPARTMENT: EEE, PROGRAM: ETT

RTQF Level, Year: BTech, YEAR 4

MODULE CODE & TITLE: GENML801 Machine learning

CREDIT UNITS: 10

SEMESTER: 1

DATE: 1st November, 2026

Group members

1. MFITUMUKIZA Deogratias

Reg no: 26RP02925

2. UZAYISENGA Marie Bella

Reg no: 26RP02449

3. NDONGOZI Chris Arsene

Reg no: 26RP00610

4. UMWIZA Alima

Reg no: 26RP01729

FRUIT DETECTION SYSTEM USING YOLOv8n AND STREAMLIT

1. Introduction

Computer vision allows computer systems to interpret images and identify objects within them. In agriculture and food handling, it can support produce sorting, stock checking and quality inspection. Manual inspection can require considerable time, and different people may not always make identical judgments.

This project describes an image-based fruit recognition application developed with the YOLOv8 object-detection framework. The system is intended to identify apples, bananas and oranges in uploaded photographs. It predicts the locations of recognized fruits and displays bounding boxes around them.

Python was used to develop the application, while Streamlit provided the browser interface. The project report records Google Colab as the model-training environment, Roboflow as the annotation review tool, GitHub for source and model management, and Streamlit Cloud for deployment.

2. Project Objectives

The main aim was to create an accessible application that can detect and classify three fruit categories in images. The specific objectives were to:

1. Prepare and validate a labelled image dataset containing apples, bananas and oranges.
2. Fine-tune a YOLOv8n object detector using the prepared dataset.
3. Assess model performance with validation and test data.
4. Build a simple web interface with Streamlit.
5. Connect the trained model to the application.
6. Store the project files in GitHub and publish the application through Streamlit Cloud.
7. Test the system with both single-fruit and mixed-fruit images.

3. Technologies and Tools

Tool	**Purpose**
Python	Main language for application development and model use.
YOLOv8n	Lightweight object detector selected for fruit recognition.
Ultralytics	Library used for model training, loading and prediction.
Google Colab	Cloud environment used for training and evaluation.
Roboflow	Reviewing and correcting fruit annotations.
OpenCV	Image-processing support.
Streamlit	Building the interactive web interface.
Git / GitHub	Version control and remote project storage.

Streamlit Cloud	Hosting the web application.

4. Dataset Preparation

The dataset was organized around three target classes, with the class identifiers recorded in the project configuration as follows:

- Class 0 – Apple
- Class 1 – Banana
- Class 2 – Orange

The images were divided into training, validation and testing subsets. Training data was used to teach the model; validation data was used to monitor learning; and the test subset was reserved for a final assessment using images not used during training. A data.yaml file specified the dataset paths and class names.

4.1 Annotation Review and Correction

For object detection, each fruit needs a bounding-box annotation that identifies both its class and its position. During inspection, some label files were found to be empty or invalid. These annotations were reviewed and corrected with Roboflow, and the revised labels were copied back into the dataset.

Dataset split	**Label files**	**Empty label files**
Training	211	0
Validation	46	0
Testing	43	0

The project record also notes that three placeholder files containing ANNOTATIONS_REQUIRED.txt were removed because they were not valid YOLO annotation files. This cleanup helped ensure that the training process used properly formatted labels.

5. Model Training with YOLOv8n

YOLOv8n was selected because it is a compact object-detection model suitable for applications where efficient inference is useful. Training took place in Google Colab using the Ultralytics library. The initial checkpoint was fine-tuned with the corrected fruit dataset.

Training setting	**Value**
Model	YOLOv8n
Number of classes	3
Image size	640 × 640
Requested epochs	30
Batch size	16

Device	GPU (device=0)
Early-stopping patience	10
Optimizer	Ultralytics default

Although 30 epochs were requested, training stopped after 11 epochs because the early-stopping condition was reached. The report gives the best checkpoint location as:

/content/fruit_training/yolov8n_fruit_corrected/weights/best.pt

The checkpoint was then downloaded and saved locally as best_corrected.pt for further testing and use in the application, according to the supplied project report.

6. Model Evaluation

The trained detector was evaluated on the test dataset. The reported measures were precision, recall, mean Average Precision at an IoU threshold of 50% (mAP50), and mean Average Precision across IoU thresholds from 50% to 95% (mAP50–95).

6.1 Overall Test Results

Metric	**Result**
Precision	0.7741
Recall	0.6951
mAP50	0.8165
mAP50–95	0.6389

The test mAP50 score of 0.8165 is equivalent to approximately 81.65%.

6.2 Results for Each Fruit Class

Fruit class	**Precision**	**Recall**	**mAP50**	**mAP50–95**
Apple	0.8134	0.7667	0.8847	0.7324
Banana	0.6933	0.5143	0.6815	0.3947
Orange	0.8155	0.8043	0.8834	0.7895

The results indicate stronger performance for apples and oranges than for bananas. Banana recall and mAP50–95 were comparatively low, suggesting that banana objects were more likely to be missed or poorly localized. Additional banana examples and more consistent annotations may improve this category.

7. Comparison of the Original and Corrected Models

The original checkpoint was compared with the fine-tuned checkpoint to examine the effect of the dataset correction and further training.

Evaluation metric	Original model	Corrected model
Precision	0.8140	0.7741
Recall	0.6997	0.6951
mAP50	0.8072	0.8165
mAP50–95	0.6737	0.6389

The corrected model produced a small increase in mAP50, from 0.8072 to 0.8165. However, its precision, recall and mAP50–95 were lower than those of the original checkpoint. The correction therefore did not improve every metric. Both checkpoints were kept during development so their predictions could be compared.

8. Development of the Streamlit Application

After training and evaluation, a web application was implemented with Streamlit. A user can upload a fruit image, choose a confidence threshold and run the detector. The application then displays the annotated image, detected fruit categories and their confidence values.

The main functions of the application are:

8. Load the YOLOv8 model checkpoint.
9. Accept an image supplied by the user.
10. Allow adjustment of the confidence threshold.
11. Pass the uploaded image to the YOLO prediction function.
12. Show bounding boxes around detected fruit objects.
13. Display the predicted classes and confidence scores.

The prediction configuration recorded in the project uses the selected confidence value and an IoU threshold of 0.45:

```
results = model.predict(\n image,\n conf=confidence,\n iou=0.45,\n verbose=False\n)
```

The confidence parameter determines the minimum score for displaying a prediction. The IoU value is used during non-maximum suppression to manage overlapping bounding boxes.

9. Integration with GitHub

GitHub was used as the remote location for the application source code and model checkpoints. The repository address supplied in the report is:

https://github.com/mugishalucky45-spec/fruit-detection-streamlit

During integration, a model-checkpoint loading error was encountered in Streamlit Cloud. The report states that the local verified checkpoint was copied to the filename expected by the application and uploaded using Git. It also records that the local and remote repositories had unrelated commit

histories. The histories were merged, the verified local model file was selected to resolve the binary conflict, and the merge was committed. The recorded successful push identifier was 970ced7.

10. Deployment with Streamlit Cloud

Streamlit Cloud was used to make the application accessible through a web browser. The deployment process involved connecting the GitHub repository, selecting app.py as the entry file, installing the required packages, loading the model checkpoint and publishing the application.

The supplied report records an initial model-file problem during deployment. It states that deployment completed after the verified model file was uploaded and the application was configured to load the matching filename. The application URL included in the report is:

https://fruit-detection-app-lxv4cm6hyhndsje8gnuzl8.streamlit.app/

Because deployment can change when repository files or model-loading code are modified, this link should be checked before submission.

11. Testing and Observations

Testing included photographs containing individual fruits and images with more than one fruit category. The application was able to produce bounding boxes and class predictions for recognized objects, but the results varied between images.

Some mixed-fruit images were processed correctly, while other examples contained missed objects, incorrect classifications or predictions with low confidence. One example in the supplied report showed an apple detected at approximately 0.89 confidence. An orange in the same image was incorrectly classified as an apple with a confidence score near 0.695, and a banana prediction appeared with a much lower score.

These observations show that adjusting the confidence threshold alone cannot eliminate classification errors. Prediction quality is also affected by the features learned during training, the amount and diversity of training data, annotation quality and the conditions in the input image.

12. Challenges Encountered and Responses

Challenge	**Response recorded in the project**
Empty annotation files	Reviewed and corrected the annotations with Roboflow.
Invalid placeholder labels	Removed the three files that were not valid YOLO labels.
Different performance across fruit classes	Evaluated each class separately and identified banana detection as a weaker area.
Incorrect predictions in mixed-fruit images	Tested additional examples and identified the need for more representative training data.
Corrupted or unsuitable model checkpoint	Replaced it with the verified local checkpoint, as

	described in the report.
Application using an incorrect model filename	Changed the model-loading filename to match the checkpoint.
Git merge conflict involving model weights	Selected the verified local checkpoint and completed the merge.
Remote repository pointing to the wrong destination	Corrected the Git remote to the intended project repository.

13. Limitations of the System

The application demonstrates the intended image-detection process, but several limitations remain:

14. The detector may not identify every fruit correctly, especially in mixed-fruit photographs.
15. Banana detection is weaker than apple and orange detection in the reported evaluation.
16. Overlapping fruit or unusual object positions can lead to missed detections.
17. Image quality, lighting, background and fruit size may influence the prediction.
18. Test-set performance does not guarantee the same results on all real-world images.
19. The current system works with uploaded images rather than a continuous live-camera stream.

The application should therefore be regarded as a prototype. Additional training and wider testing would be necessary before using it in a commercial environment.

14. Recommendations for Future Improvement

- Collect more banana images showing different varieties, orientations and backgrounds.
- Add more photographs containing different combinations of the three fruit categories.
- Improve the consistency and accuracy of bounding-box annotations.
- Use data augmentation to expose the model to different lighting and background conditions.
- Compare the original and corrected checkpoints on the same test images.
- Select confidence and IoU thresholds through a consistent evaluation process.
- Test the application with a wider range of real-world photographs.
- Consider adding live-camera detection in a future version.
- Measure inference speed and resource use if the system is deployed on an edge device.

15. Conclusion

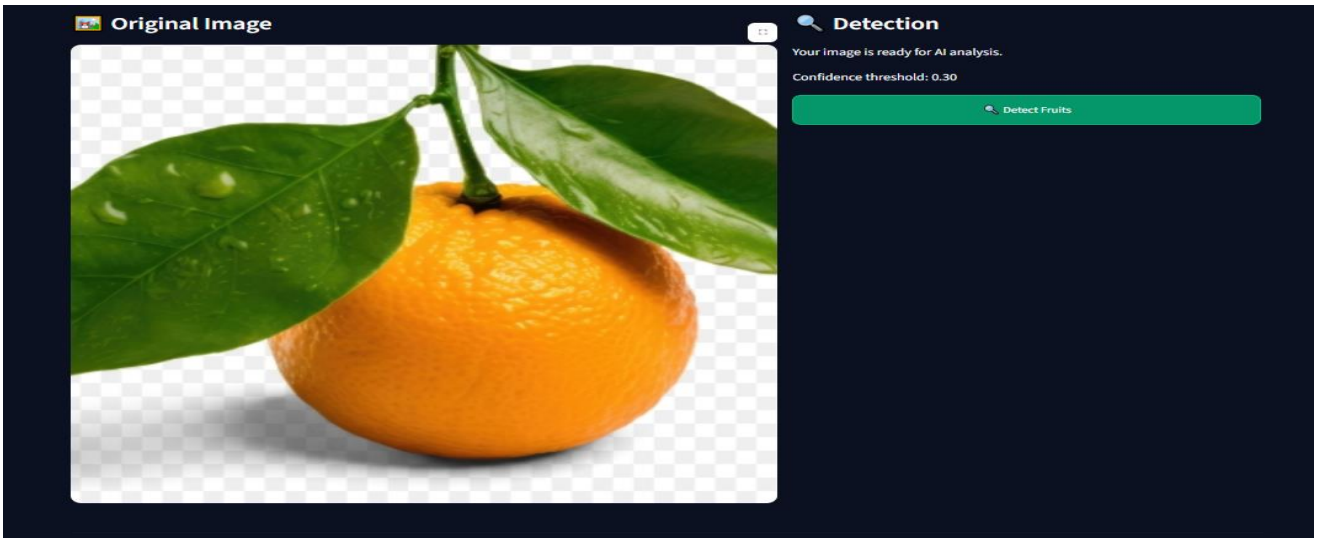
This project combined deep-learning object detection with web application development to produce a fruit-recognition prototype using YOLOv8n and Streamlit. The work covered dataset preparation and annotation correction, model fine-tuning, performance evaluation, interface development, GitHub integration and cloud deployment.

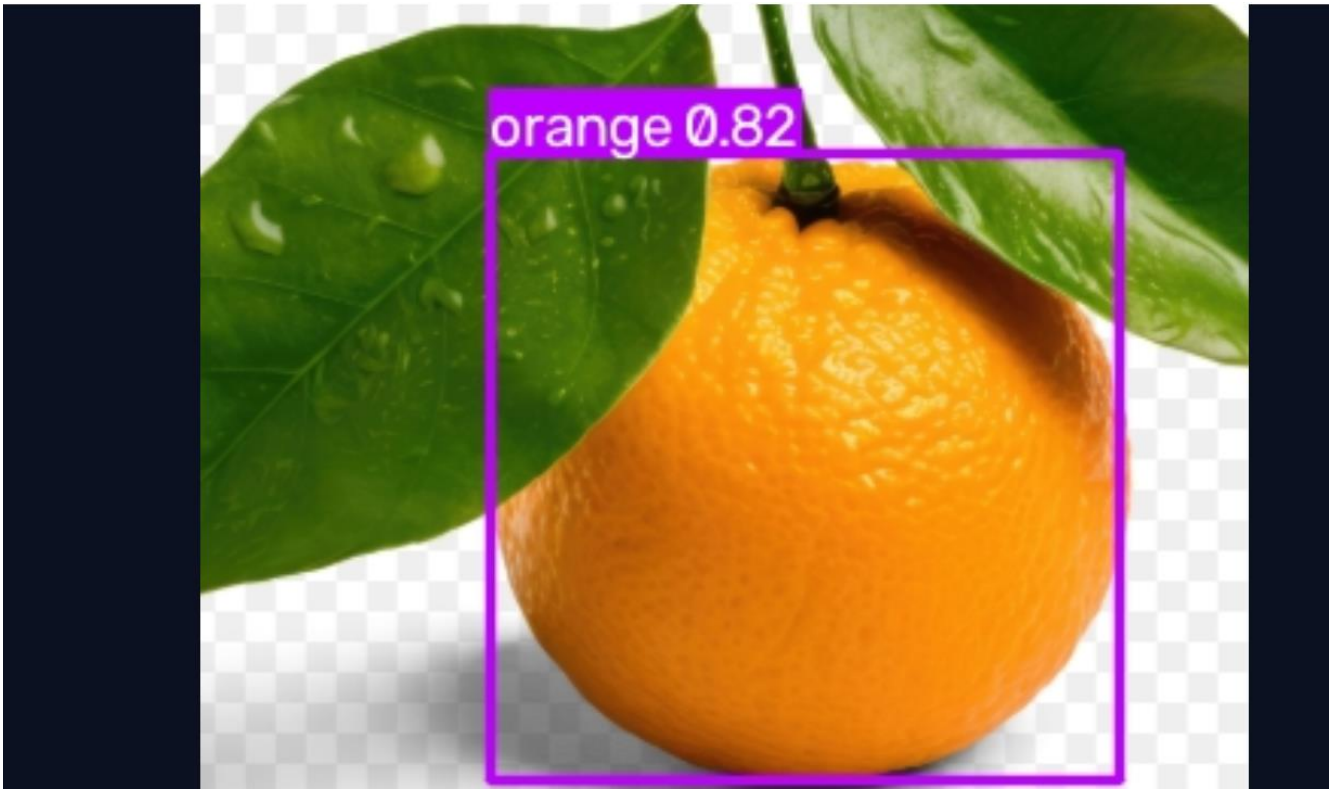
On the reported test set, the corrected model achieved precision of 77.41%, recall of 69.51%, mAP50 of 81.65% and mAP50–95 of 63.89%. Apples and oranges achieved stronger class-level results than

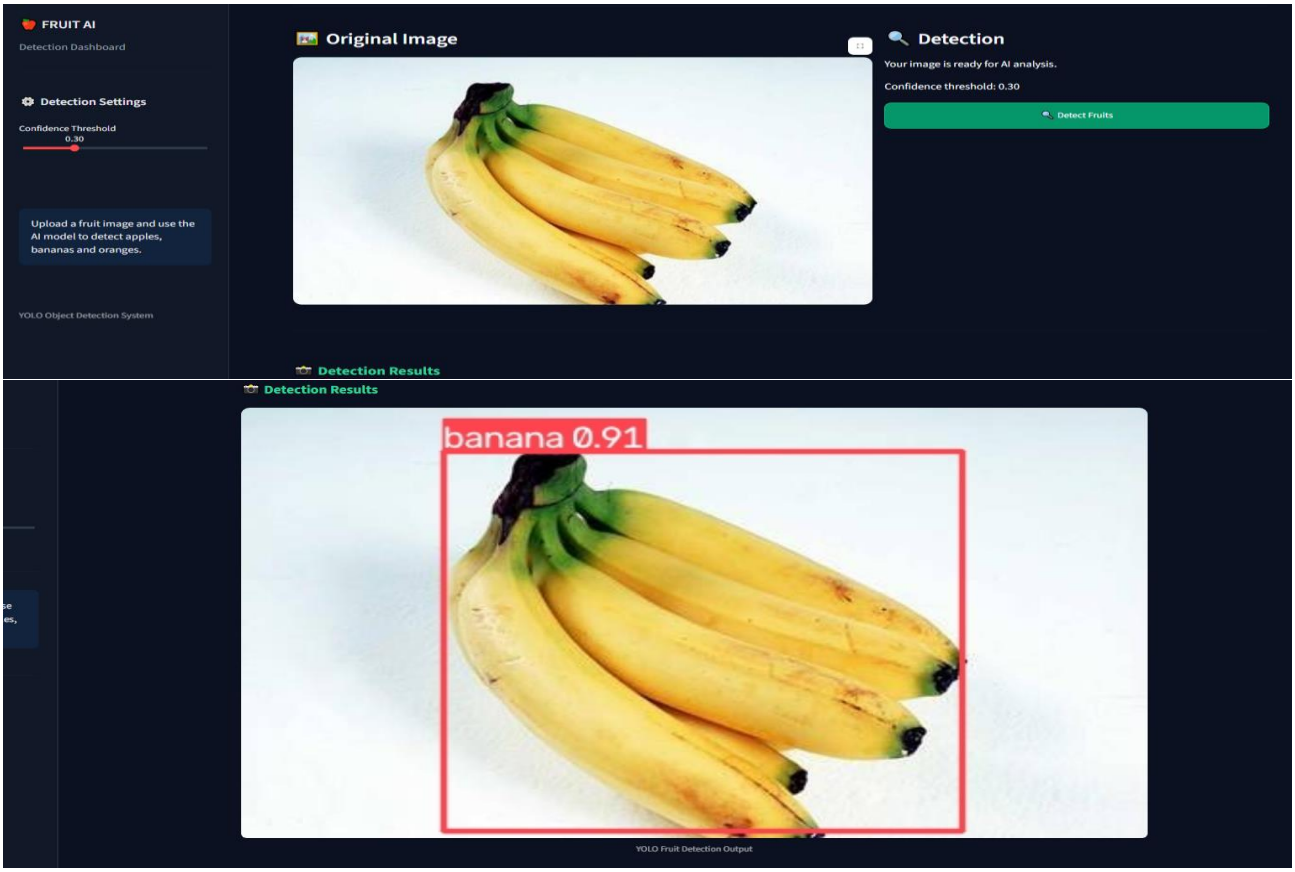
bananas. Comparison with the original model showed a modest improvement in mAP50, but not in every evaluation measure.

The application allows users to submit images and view predicted fruit labels, confidence values and bounding boxes. Some classification errors remain, particularly in mixed-fruit images and banana detection. Further dataset improvement, model training and testing are recommended to increase reliability.

16. Test Results and Application Screenshots







17. References

Roboflow. (n.d.). Roboflow documentation. https://docs.roboflow.com/

Project link to Github: https://github.com/mfitumukizadeo-jpg/Fruit-Detection.git

Project link to streamlit: https://fruit-detection-gj59wwqg49qnavnarv7jrh.streamlit.app/